

Vinci Unitree Course — Day 7 Lecture Notes

G1 Audio, Speech, LED, and Capstone Integration

Course: Vinci Unitree Robotics Training

Session: Day 7 — G1 Audio, Speech, LED, and Capstone

Duration: 3 hours

Primary platform: Unitree G1 humanoid

Prepared by: Manus AI for Vinci AI

Repository basis: Vinci-AI-Analytics/vinci-unitree , course/day-07/

Copyright: © Vinci AI. For internal training use.

1. Session Purpose and Teaching Frame

Day 7 is the course's **human-facing integration day**. Days 5 and 6 introduced the G1 as a DDS-connected humanoid platform, established low-state observability, practiced safe locomotion, and introduced arm actions and integration rules. Day 7 adds the robot's expressive interface: **audio, text-to-speech, chest RGB LEDs, and a capstone-style demonstration that combines speech, light, and supervised motion**.

The central message for learners is that a humanoid robot becomes much easier to understand when it can communicate its state. A walking or gesturing robot without communication is difficult for bystanders to interpret. A robot that speaks before moving, changes LEDs to show state, waits between actions, and logs return codes is more inspectable, more teachable, and safer to operate. Day 7 therefore treats audio and LEDs not as decorative features, but as **operator-facing observability channels**.

Teaching definition: In this lecture, multimodal interaction means a controlled sequence in which the G1 combines **spoken output, visible LED status, and preapproved physical motion** while maintaining a single DDS command owner, a clear safety envelope, and a human supervisor.

The official Day 7 course README names the session **"G1 Audio, Speech, LED & Capstone"** and lists four labs: readiness, audio playback, TTS/LED debrief, and a capstone that links audio with Day 6 walking and arm actions. The README explicitly states that dexterous hand control is out of scope; capstone manipulation refers to Day 6 arm gestures, not grasping or hand control.

Day 7 theme	Meaning for the instructor	Meaning for learners
Audio as status	Use speech to announce intent and completion.	Do not surprise bystanders with silent movement.
LEDs as state	Use RGB colors to make execution phase visible.	Associate colors with readiness, action, completion, or fault.
Motion as supervised action	Permit only approved gestures or Day 6 motion fragments.	Do not improvise unsafe locomotion or arm trajectories.
Capstone as integration	Combine previous modules without creating command conflicts.	Demonstrate sequencing, timing, and evidence collection.

By the end of the three-hour session, learners should be able to explain how the G1 audio client fits into the same DDS-based SDK architecture used earlier in the course, run the Day 7 audio and LED scripts safely, interpret return codes and timing constraints, and design a short capstone sequence that communicates what the robot is doing before, during, and after physical actions.

2. Three-Hour Lecture Plan

The recommended pacing below assumes a mixed lecture-and-lab format. The instructor should adjust the live demonstration time according to robot availability, network stability, student experience, and room safety conditions. If only one G1 is available, use the lecture sections to keep the group engaged while one team operates the robot under supervision.

Time	Segment	Instructor objective	Learner outcome
0:00–0:15	Opening and Day 6 readiness gate	Reconnect Day 7 to DDS, G1 readiness, and Day 6 motion safety.	Learners can explain why audio/LED work still requires low-state readiness.
0:15–0:35	G1 audio and lighting hardware	Introduce speaker, microphone array, RGB light strip, and official <code>AudioClient</code> capabilities.	Learners understand why Day 7 uses <code>AudioClient</code> for both speech and LEDs.
0:35–1:00	DDS session ownership and script anatomy	Walk through <code>ChannelFactoryInitialize</code> , <code>AudioClient</code> , <code>LocoClient</code> , timeouts, and <code>rt/lowstate</code> checks.	Learners can read the Day 7 scripts and identify the safety gates.
1:00–1:25	Lab 1: audio client and volume/TTS	Demonstrate <code>GetVolume</code> , <code>SetVolume</code> , and <code>TtsMaker</code> .	Learners can run the audio script and interpret success/error codes.
1:25–1:35	Break and robot reset	Return to safe idle state, confirm volume and LED status.	Learners see that resets are part of responsible robotics practice.
1:35–2:00	Lab 2: LED and gesture debrief	Explain <code>LedControl</code> , RGB values, timing, and <code>WaveHand()</code> as a physical action.	Learners can map LED colors to robot state and understand gesture timing.
2:00–2:30	Capstone design pattern	Present a safe state-machine pattern for voice + LED + gesture + optional Day 6 arm action.	Learners can design a bounded capstone sequence without command-path conflicts.
2:30–2:50	Troubleshooting and evidence	Diagnose common DDS, service, volume, TTS, LED, and concurrency failures.	Learners can capture logs, return codes, and observed behavior.

Time	Segment	Instructor objective	Learner outcome
2:50– 3:00	Assessment and Day 8 transition	Conduct knowledge checks and collect student capstone plans.	Learners can articulate safe integration rules and next steps.

The instructor should repeatedly emphasize that Day 7 is not a license to combine every capability at once. It is a controlled integration exercise. The desired professional behavior is to **sequence one capability at a time, announce intent, wait for completion, observe the robot, and only then proceed.**

3. Day 7 Repository Structure and Lab Intent

The Day 7 folder is intentionally compact. It points learners toward a core audio-client example and a set of teaching variants that make the underlying service calls easier to inspect. This structure supports two types of teaching. First, the instructor can run the wired example to show the intended behavior quickly. Second, the instructor can open the more verbose teaching scripts to explain how each call works.

Repository item	Role in the lecture	Notes for delivery
<code>course/day-07/README.md</code>	Day overview and scope boundary.	Confirms platform, prerequisite, lab sequence, and dexterous-hand exclusion.
<code>course/day-07/lab-00/README.md</code>	Readiness gate.	Requires Day 6 completion and a single DDS session owner before capstone.
<code>course/day-07/lab-01/README.md</code>	Audio-client lab entry point.	Runs <code>lab-01-02_g1_audio_client_example.py</code> with the network interface.
<code>course/day-07/lab-02/README.md</code>	TTS/LED debrief.	Uses the same script but focuses discussion on <code>TtsMaker</code> , <code>LedControl</code> , and <code>WaveHand()</code> .
<code>course/day-07/lab-03/README.md</code>	Capstone concept.	Links audio/TTS/LED with Day 6 arm and integration labs; glue script is marked TBD.
<code>course/day-07/lab-01-02_g1_audio_client_example.py</code>	Wired Thomas example.	Demonstrates volume probing, TTS, <code>WaveHand</code> , and RGB LED color cycling.
<code>course/day-07/lab-01/lab01_audio.py</code>	Teaching script for infrastructure.	Prints service constants, API IDs, raw RPC payloads, and optional helper calls.
<code>course/day-07/lab-01/lab01_audio_client.py</code>	Expanded combined demo.	Adds readiness checks, dry-run behavior, custom text, volume clamping, and LED/TTS sequencing.
<code>course/day-07/lab-01/lab01_audio_only.py</code>	Narrow audio-only utility.	Useful when motion should be disabled or room conditions are constrained.
<code>course/day-07/lab-01/lab01_led_only.py</code>	Narrow LED-only utility.	Useful for teaching RGB control without speech or gesture.

The Day 7 capstone folder intentionally references Day 6 assets rather than duplicating them. It points to Day 6 arm action and integration scripts because the course's model is incremental: Day 5

established G1 architecture and DDS safety; Day 6 introduced motion and arm action; Day 7 adds audio and visible state and then asks learners to combine those modules carefully.

4. G1 Audio and Lighting Hardware Context

Official Unitree documentation explains that G1 includes upper-body audio and lighting hardware for voice interaction. The documented facilities include an RGB light strip with 256 colors, a speaker rated at 8 ohms and 3 W with 5 W peak, and a four-microphone array with 20 mm microphone spacing.¹ These hardware elements are important because they provide both output and potential input channels for human-robot interaction.

For Day 7, the primary focus is **output**: volume, TTS speech, audio playback concepts, and RGB LED signaling. The repository's scripts do not implement a full ASR-driven command loop. This is intentional. A voice-command robot is compelling, but a classroom environment with multiple people speaking, moving, and reacting is a poor place to introduce uncontrolled speech-triggered motion. Instead, Day 7 teaches the safer half of the interaction loop first: the robot announces what it is doing, shows visual state, and performs a bounded supervised action.

Hardware capability	Official description	Day 7 use
Speaker	G1 includes speaker hardware for voice interaction. ¹	Text-to-speech announcements and playback status.
RGB light strip	Unitree documents RGB light strip control and 256-color lighting capability. ¹	State indication: ready, active, success, or fault.
Microphone array	G1 includes a four-microphone array. ¹	Discussed as future ASR input; not required for Day 7 capstone.
Development computing unit	G1-EDU includes a development computing unit for secondary development. ²	Runs SDK scripts that communicate with robot services.
Remote/debug workflow	Unitree quick-start guidance describes damping, ready, operation, and debug workflows. ³	Reinforces safe operating state before SDK-driven demonstrations.

The instructor should be clear that **audio is not merely sound**. In robotics, sound is a communication channel. A professional robot should announce transitions such as “starting demonstration,” “changing LED color,” “performing wave,” or “sequence complete.” These announcements make system behavior legible to observers and help the operator detect whether the script has progressed as expected.

5. The `AudioClient` Service Model

The Day 7 scripts use `AudioClient`, imported from `unitree_sdk2py.g1.audio.g1_audio_client`. Official Unitree documentation describes `AudioClient` as the class that implements text-to-speech, audio control/playback, and lighting control functions.¹ This is the key architectural point: **LED control lives through the audio client**, because the RGB strip is part of the upper-body interaction hardware rather than a separate course-level LED service.

Unitree's documented `AudioClient` interfaces include `TtsMaker`, `GetVolume`, `SetVolume`, `LedControl`, `PlayStream`, and `PlayStop`.¹ The Day 7 repository directly exercises `GetVolume`, `SetVolume`, `TtsMaker`, and `LedControl`. It also teaches the existence of ASR and stream playback through constants and API labels in `lab01_audio.py`, but those are not required for the core 3-hour practical.

Function	Conceptual purpose	Parameter pattern	Day 7 interpretation
<code>GetVolume()</code>	Read current system volume.	No user payload in helper form.	A probe used before and after volume changes.
<code>SetVolume(volume)</code>	Set system volume.	Volume in the range 0–100. ¹	Use conservative classroom values; clamp unsafe inputs.
<code>TtsMaker(text, speaker_id)</code>	Convert text to speech.	Text plus speaker role ID. ¹	Main speech-output primitive.
<code>LedControl(R, G, B)</code>	Set RGB light-strip color.	Each color channel ranges 0–255. ¹	State visualization primitive.
<code>PlayStream(...)</code>	Play PCM stream audio.	Application name, stream ID, PCM bytes. ¹	Mention as future extension, not required for Day 7.
<code>PlayStop(app_name)</code>	Stop stream playback.	Application name. ¹	Mention as future extension and cleanup tool.

Unitree documents that `TtsMaker` returns `0` on success and otherwise returns an error code.¹ The same return-code thinking applies throughout the course: a script should not treat service calls as magical commands that always work. It should print return codes, wait after long-running actions, and

report nonzero results. Day 7’s teaching scripts follow this pattern through `_run_step()` , which prints each step name and its return code.

6. Speaker IDs, Language Boundaries, and TTS Discipline

A practical Day 7 detail is the `speaker_id` . Unitree documents `speaker_id` 0 for Chinese roles and `speaker_id` 1 for English roles, and notes that mixed Chinese and English modes are not supported.¹ The course repository reflects this reality in two ways. The wired Thomas example uses Chinese phrases with speaker ID 0. The audio-only utility defaults to an English phrase and sets `SPEAKER_ID = 1` .

This distinction matters pedagogically because speech demos often fail for avoidable reasons. Learners may enter an English phrase but leave the Chinese speaker ID, or mix English and Chinese in the same sentence. The result may be wrong pronunciation, unexpected silence, or confusing behavior. The lecture should therefore introduce a simple rule: **choose one language mode per TTS call and match the speaker ID to that language.**

TTS choice	Recommended Day 7 practice	Reason
Chinese phrase	Use <code>speaker_id=0</code> .	Matches Unitree’s documented Chinese role mapping. ¹
English phrase	Use <code>speaker_id=1</code> .	Matches Unitree’s documented English role mapping. ¹
Mixed-language sentence	Avoid in Day 7.	Unitree notes mixed Chinese/English modes are not supported. ¹
Very long phrase	Keep short and wait for completion.	Short utterances are easier to verify and less disruptive.
Loud volume	Avoid; use moderate values.	Classroom demos should prioritize clarity and comfort.

A strong instructor demonstration is to show the default phrase first, then run a short custom phrase such as “Day seven audio lab is ready.” The goal is not theatrical speech; it is reliable service invocation and observable system state.

7. DDS Readiness and Single Session Ownership

Day 7 may feel less dangerous than Day 6 because it focuses on audio and LEDs, but the same communication discipline still applies. The scripts initialize the DDS communication layer using

`ChannelFactoryInitialize(0, interface)` , then create service clients. Several teaching scripts also wait for the `rt/lowstate` topic before proceeding. This wait is a readiness test: the computer must be able to receive state from the robot before it should send service calls.

The `lab01_audio.py` , `lab01_audio_client.py` , `lab01_audio_only.py` , and `lab01_led_only.py` scripts all include an `_init_dds()` pattern. If `CYCLONEDDS_URI` is set, the script warns that interface-only initialization may be bypassed; otherwise, it initializes DDS with the requested network interface. The scripts then subscribe to `rt/lowstate` and wait up to five seconds for a message. If no low-state data arrives, they fail early and tell the learner to run the connection check.

Readiness check	What it proves	Failure meaning	Instructor response
<code>CYCLONEDDS_HOME</code> is set	SDK environment is configured.	CycloneDDS or environment setup is incomplete.	Return to Day 5 setup workflow.
Correct network interface	DDS binds to the robot-facing NIC.	Robot topics/services may be unreachable.	Run interface discovery and connection check.
<code>rt/lowstate</code> received	The workstation can hear robot state.	Network, interface, robot power, or DDS issue.	Do not run action scripts until fixed.
One script owns the session	Command conflicts are avoided.	Multiple clients may send overlapping commands.	Stop extra terminals and assign one operator.
Operator and E-stop ready	Human safety layer is present.	Demonstration may be unmanaged.	Pause until role assignment is clear.

The Day 7 README explicitly instructs teams to confirm Day 6 completion and maintain a **single DDS session owner** before capstone. This should be treated as a formal gate. The person running the terminal is the command owner. Others may observe, record, or call out safety concerns, but they should not run simultaneous robot-control scripts.

8. Lab 0 — Readiness Before Interaction

Lab 0 is short, but it is one of the most important parts of the day. The goal is not to write new code; the goal is to decide whether the robot, room, network, and team are ready for an interaction demo. Day 7 combines spoken output and visible status with at least one physical gesture, so the room must be prepared.

The instructor should begin by asking each team to confirm four items. First, Day 6 locomotion and arm-control labs must be complete. Second, the robot must be in the approved operating/debug state for SDK work. Third, only one terminal should be authorized to send commands. Fourth, the operator must know how to stop, damp, or return the robot to a safe state according to the local training policy and Unitree quick-start guidance.

Lab 0 item	Pass criterion	Evidence to collect
Day 6 completion	Team can explain Day 6 readiness and motion safety.	Verbal check or Day 6 log.
Environment	<code>unitree_env</code> or equivalent SDK environment is active.	Terminal prompt and package imports.
DDS	Correct interface chosen and <code>rt/lowstate</code> is visible.	Connection check output.
Session ownership	One operator terminal selected.	Team role assignment.
Motion envelope	Clear space, robot supervised, remote/E-stop ready.	Instructor visual confirmation.
Script scope	Team knows which script will run and what it will do.	Pre-run explanation by student.

Instructor prompt: “Before we let the robot speak or move, tell me which process owns DDS, which network interface you are using, what the script will do first, and how you will stop the demonstration if the robot behaves unexpectedly.”

This gate helps students internalize that safe robotics begins before the first command. It also reduces troubleshooting time because many Day 7 failures are not audio failures; they are environment, interface, or session-ownership failures.

9. Lab 1 — Audio Client, Volume, and TTS

Lab 1 introduces the main audio-client workflow. The repository’s README shows the basic command pattern:

```
conda activate unitree_env
cd "$(git rev-parse --show-toplevel)"
python course/day-07/lab-01-02_g1_audio_client_example.py eth0
```

The interface name should be replaced with the correct local robot-facing interface. On some machines this might be `eth0` ; on others it may be a macOS-style interface such as `en6` , or a Linux name such as `enp0s31f6` . The scripts also allow an environment variable pattern in some teaching variants, but the safest classroom habit is to **state the interface explicitly**.

The core sequence in the wired example is straightforward. The script initializes DDS, creates an `AudioClient` , creates a `LocoClient` , probes volume, sets volume to 85, probes volume again, performs a hand wave, plays an intro TTS phrase, announces the LED test, cycles red, green, and blue LEDs, waits, and plays an outro phrase. The teaching version `lab01_audio.py` breaks the same ideas into smaller concepts by printing API constants and showing the raw JSON payload shapes.

Step	Script behavior	Teaching point
1	Initialize DDS on the selected interface.	Communication must bind to the correct network adapter.
2	Create and initialize <code>AudioClient</code> .	Service client must be initialized before calls.
3	Optionally wait for <code>rt/lowstate</code> .	Readiness should precede action.
4	Call <code>GetVolume()</code> .	Always measure current state before changing it.
5	Call <code>SetVolume(85)</code> or a chosen value.	Use bounded, comfortable values.
6	Call <code>TtsMaker(text, speaker_id)</code> .	TTS is a service call with language-role selection.
7	Wait for speech to finish.	Speech is asynchronous enough that scripts need timing margins.

The lecture should emphasize the difference between **helper methods** and **raw RPC calls**. In helper form, a learner writes `audio.TtsMaker(text, speaker_id)` . In raw form, `lab01_audio.py` shows that calls can be mapped to an API ID and a JSON parameter dictionary. The helper is easier to use, but the raw view is valuable because it shows the actual service design: each call has a name, parameters, a return code, and a timeout.

10. Understanding `lab01_audio.py` as an Infrastructure Lesson

The `lab01_audio.py` script is especially useful for lecture because it turns the audio demo into an inspection exercise. It imports `AUDIO_SERVICE_NAME`, `AUDIO_API_VERSION`, and API IDs from `unitree_sdk2py.g1.audio.g1_audio_api`. It then constructs a readable map from API IDs to concepts such as TTS, ASR, start PCM stream play, stop stream play, get volume, set volume, and RGB LED.

This script is useful even without running the robot. In `--show-api` mode, it can print the catalog and exit without a robot connection. That gives the instructor a safe way to explain service architecture before live operation. It also helps learners understand that SDK wrappers are not mysterious; they are structured clients around known service endpoints and parameter payloads.

<code>lab01_audio.py</code> component	Why it matters
<code>API_ID_LABELS</code>	Converts numeric API IDs into readable concepts.
<code>EmptyParams</code>	Shows that some calls still send an empty JSON object.
<code>VolumeParams</code>	Shows how <code>SetVolume</code> becomes <code>{"volume": N}</code> .
<code>TtsParams</code>	Shows how TTS becomes <code>{"index": ..., "text": ..., "speaker_id": ...}</code> .
<code>_wait_lowstate()</code>	Prevents action calls when robot state is not visible.
<code>--dry-run</code>	Lets teams initialize clients without changing volume or speaking.
<code>--skip-tts</code>	Allows volume testing without speech output.
<code>--use-client-helpers</code>	Allows comparison between raw calls and helper methods.

The instructor can use this script to ask learners a diagnostic question: “If `GetVolume` works but `TtsMaker` fails, what does that tell you?” A good answer is that DDS and the audio service may be reachable, but the specific TTS API, language role, service version, text payload, or timeout may still be wrong. This encourages targeted debugging rather than blaming “the robot” generically.

11. Lab 2 — RGB LEDs and Gesture Debrief

Lab 2 uses the same main script but changes the lens. Instead of focusing on volume and speech, the instructor focuses on `LedControl` and `WaveHand()`. This is a powerful moment because it shows the boundary between **expressive state output** and **physical motion**.

Unitree documents `LedControl(R, G, B)` with red, green, and blue parameters in the range 0–255, and notes that the interval between calls must be greater than 200 ms.¹ The repository scripts

use one-second or 1.5-second waits between LED color changes, which is conservative and classroom-friendly. These waits make the sequence visible to learners and avoid violating the documented minimum interval.

LED color	Suggested classroom meaning	Example call
Red	Attention, warning, or action in progress.	<code>audio.LedControl(255, 0, 0)</code>
Green	Ready, safe, or successful completion.	<code>audio.LedControl(0, 255, 0)</code>
Blue	Demo mode, information, or standby.	<code>audio.LedControl(0, 0, 255)</code>
Off	Sequence complete or reset.	<code>audio.LedControl(0, 0, 0)</code>

The `WaveHand()` action is physically different from an LED call. The script obtains it through `LocoClient`, not `AudioClient`, and the robot's body must be supervised because a gesture involves actual motion. Even a simple wave can surprise observers if it happens without warning. Therefore, the recommended sequence is to speak first, wait briefly, perform the wave, wait for completion, then speak or change LEDs again.

Professional sequencing rule: Do not move first and explain later. In human-facing robotics, the robot should announce, indicate, move, settle, and then confirm completion.

A good Lab 2 debrief asks students to explain why LED timing matters, why `WaveHand()` belongs in the safety discussion, and how they would modify the script so that LEDs communicate phases of the action rather than merely cycling colors.

12. ASR and Audio Streams as Future Extensions

The official Unitree audio documentation includes more than Day 7 uses. It describes ASR messages available on `rt/audio_msg`, with fields such as recognized text, angle, speaker ID, confidence, language, and an `is_final` flag.¹ It also documents `PlayStream` and `PlayStop` for PCM audio playback.¹

These capabilities should be mentioned, but not overemphasized. Day 7 is not a production voice-command course. The safe instructional path is to teach **robot-to-human output** before **human-to-robot input**. Speech recognition introduces ambiguity: multiple people may speak, microphones may pick up background noise, recognized text may be incomplete, and unsafe actions must never be triggered by uncertain natural language in a classroom.

Capability	Why it is interesting	Why it is not core Day 7
ASR on <code>rt/audio_msg</code>	Enables future voice-command interfaces.	Requires command filtering, confidence thresholds, and safety gating.
Audio stream playback	Enables custom audio clips and richer feedback.	Requires correct PCM format and playback management.
Play stop	Allows cleanup of stream playback.	Not needed for short TTS phrases.
Microphone direction/angle	Could support human-aware interaction.	Not needed for supervised capstone output sequence.

The instructor can frame ASR as a future project: “Today we make the robot explain itself. A later system may listen to humans, but listening must be paired with strict intent validation and a physical safety policy.” This keeps learners excited while preserving the discipline of the training.

13. Capstone Integration Pattern

The Day 7 capstone is described as **voice + walk + arm**, but the repository notes that the glue script is still TBD. Therefore, the lecture should not pretend that a finished integrated script exists. Instead, it should teach a safe design pattern that learners can use to plan or implement the capstone when the instructor authorizes it.

A robust Day 7 capstone should be a small state machine. Each state has a clear purpose, a visible or audible signal, a bounded physical action if any, a wait or observation period, and a return-code check. The capstone should not run audio, locomotion, and arm commands concurrently from separate scripts. It should also avoid dexterous hand control, because the Day 7 README explicitly marks hand control out of scope.

Capstone state	Example behavior	Safety principle
READY_CHECK	Confirm <code>rt/lowstate</code> , operator role, clear space.	No action before readiness.
ANNOUNCE_START	TTS: “Starting Day 7 capstone.” LED blue.	Observers know what is happening.
GESTURE	<code>WaveHand()</code> or approved Day 6 arm action.	One physical action at a time.
STATUS_UPDATE	LED green or TTS confirmation.	Robot communicates completion.
OPTIONAL_MOTION	Only instructor-approved Day 6 locomotion fragment.	No new untested motion on capstone day.
SHUTDOWN_SIGNAL	LED off or green, TTS: “Demo complete.”	Return to a known safe state.
LOG_RESULTS	Record return codes and observations.	Evidence supports assessment and debugging.

The capstone should be assessed not by how flashy it is, but by how well it demonstrates system discipline. A team that performs a short, predictable, well-logged TTS + LED + wave sequence has learned more professionally than a team that improvises an unstable walking demonstration.

14. Suggested Safe Capstone Script Outline

The following pseudocode is not a replacement for instructor-approved repository code. It is a teaching outline showing how Day 7 concepts should be sequenced. The important idea is that every step is named, bounded, and checked.

```

initialize_dds(interface)
require_lowstate(timeout_s=5.0)

audio = AudioClient()
audio.SetTimeout(10.0)
audio.Init()

loco = LocoClient()
loco.SetTimeout(10.0)
loco.Init()

run_step("set LED blue", lambda: audio.LedControl(0, 0, 255))
run_step("announce start", lambda: audio.TtsMaker("Day seven capstone starting
time.sleep(4)

run_step("wave hand", loco.WaveHand)
time.sleep(4)

run_step("set LED green", lambda: audio.LedControl(0, 255, 0))
run_step("announce complete", lambda: audio.TtsMaker("Capstone complete.", 1))
time.sleep(3)

run_step("LED off", lambda: audio.LedControl(0, 0, 0))

```

A stronger production version would add exception handling, explicit state logging, a final cleanup block, and a configuration file for phrases, LED colors, and wait times. However, the pseudocode is sufficient to teach the integration mindset: **initialize once, own the session, call one capability at a time, wait, check, and log.**

15. Command-Path Mixing Policy

The course has repeatedly introduced command-path discipline because Unitree robots have multiple ways to receive commands: built-in controllers, remote-control modes, SDK service clients, high-level locomotion clients, arm SDK streams, and audio services. Day 7 adds another layer: speech and LEDs can make the robot feel more autonomous than it really is. That psychological effect can lead teams to over-combine scripts.

Unitree's G1 quick-start documentation warns that the built-in motion control program may periodically send commands, and that SDK development/debugging should use debug mode to avoid conflicting instructions and potential jitter.³ This warning should be connected directly to Day 7. If a team runs an audio script that also calls `WaveHand()` while another terminal runs an arm sequence, the problem is not just software cleanliness; it is physical command conflict.

Mixing case	Allowed in Day 7?	Reason
One script uses <code>AudioClient</code> only	Yes, after readiness.	No physical motion except sound/light.
One script uses <code>AudioClient</code> plus <code>WaveHand()</code>	Yes, under supervision.	Physical gesture is bounded and expected.
One integrated script sequences audio and Day 6 arm action	Instructor-approved only.	Must avoid simultaneous arm/locomotion command paths.
Multiple terminals run separate robot scripts	No.	Violates single DDS session ownership.
Voice recognition triggers movement automatically	No for Day 7 core lab.	Requires additional intent validation and safety architecture.
Dexterous hand manipulation	No.	Explicitly out of scope in the Day 7 README.

A useful teaching phrase is: “**Integration does not mean concurrency.**” In a capstone, integration means that different capabilities cooperate in a planned sequence. It does not mean that every script is launched at once.

16. Troubleshooting Guide

Day 7 problems usually fall into five categories: environment setup, DDS visibility, audio service reachability, language/volume configuration, and timing/concurrency. The instructor should make students diagnose systematically instead of rerunning commands blindly.

Symptom	Likely cause	Diagnostic question	Corrective action
Script exits because <code>CYCLONEDDS_HOME</code> is not set.	SDK environment not activated or setup not sourced.	Is <code>unitree_env</code> active and CycloneDDS installed?	Activate environment and export <code>CYCLONEDDS_HOME</code> .
No <code>rt/lowstate</code> within five seconds.	Wrong interface, robot not reachable, DDS configuration issue.	Which network interface is connected to the robot?	Run Day $\frac{5}{6}$ connection check and correct interface.
<code>GetVolume</code> returns unexpected result.	Audio service unavailable or response format differs.	Did <code>AudioClient.Init()</code> complete?	Recheck service version, timeout, and robot state.
TTS does not sound.	Volume too low, wrong speaker ID, service error, language mismatch.	What return code was printed? Which speaker ID was used?	Set moderate volume, match language, keep phrase short.
LED does not change.	RGB call failed or timing too fast.	Are return codes nonzero? Are calls spaced out?	Use one-second waits and verify <code>LedControl</code> return.
Wave occurs unexpectedly or at wrong time.	Script sequence misunderstood or multiple clients active.	Is another terminal running?	Stop all extra scripts and restore single owner.
Robot jitters during SDK work.	Built-in controller or other command path conflicts.	Is robot in the appropriate debug/SDK mode?	Follow Unitree quick-start debug guidance. 3
Demo is too loud or disruptive.	Volume selected for quiet lab but not classroom.	Is the volume appropriate for the room?	Lower volume and warn before playback.

The most important troubleshooting habit is to preserve evidence. Students should copy command output, return codes, selected interface, script name, phrase, volume, speaker ID, and observed robot behavior. Without those details, the instructor cannot distinguish a language issue from a DDS issue or a service issue.

17. Instructor Demonstration Script

A polished 15-minute instructor demonstration can follow this sequence. First, show the Day 7 README and point out the lab sequence and the out-of-scope note about dexterous hand control. Second, open `lab01_audio.py` and identify the imported constants and API labels. Third, run `--show-api` if available to show the service catalog without touching the robot. Fourth, run a dry run on the correct interface. Fifth, run the audio-only or full audio-client example with a short phrase and conservative volume.

The instructor should narrate every step. This is not just presentation style; it models professional robot operation. A good narration might be: “The robot is ready. I am the only session owner. I will set the LED blue, play one short phrase, wait for completion, then perform a wave. If anything unexpected happens, we stop and return to a safe state.”

Demo stage	Recommended command or action	What students should observe
Inspect files	Open Day 7 README and lab script.	Scope and sequence are visible before execution.
Show API	<code>python .../lab01_audio.py <iface> --show-api</code>	Audio service calls are named and inspectable.
Dry run	<code>python .../lab01_audio_client.py <iface> --dry-run</code>	Clients initialize without speech, LED, or motion.
Audio-only	<code>python .../lab01_audio_only.py <iface> --text "Day seven audio is ready."</code>	TTS and volume work without gesture.
LED-only	<code>python .../lab01_led_only.py <iface></code>	RGB colors cycle and return off.
Full demo	<code>python .../lab01_audio_client.py <iface> --text "Day seven demo starting."</code>	Speech, wave, and LEDs follow a predictable sequence.

If the class is large, the instructor can ask students to fill out a simple observation table during the demo: what was announced, what color appeared, what movement occurred, what return codes were printed, and what wait periods were used.

18. Student Activity: Design a Day 7 Interaction Contract

After the live demonstration, each team should design an **interaction contract** before running any capstone-like sequence. This contract is a short written agreement between the team and the robot: what the robot will say, what colors it will display, what physical actions it will perform, what it will not do, and how the team will stop or reset the sequence.

Contract field	Example answer
Operator	"Student A owns the terminal and runs all commands."
Interface	"Robot network interface is en6."
Initial phrase	"Starting Day 7 capstone demonstration."
LED mapping	"Blue = starting, red = action, green = complete, off = reset."
Physical action	"One WaveHand() only; no walking."
Waits	"Four seconds after TTS, four seconds after wave, one second between LEDs."
Stop condition	"Any unexpected motion, nonzero return code, or instructor stop call."
Evidence	"Save terminal output and note observed robot behavior."

This activity transforms Day 7 from a script-running exercise into a systems-design exercise. It also prepares students for real robotics work, where a demonstration plan, safety case, and evidence log are often more important than a single impressive action.

19. Assessment Questions and Expected Answers

The following questions can be used for oral assessment, written checkouts, or team debrief. They are designed to test conceptual understanding rather than memorization.

Question	Expected answer
Why does Day 7 still require <code>rt/lowstate</code> readiness if the main focus is audio?	Because the same DDS and robot connectivity assumptions apply, and some scripts also include physical actions such as <code>WaveHand()</code> . State visibility is a safety gate.
Why does the LED script use <code>AudioClient</code> instead of a separate LED client?	The G1 upper-body audio and lighting functions are grouped under the audio interaction hardware and exposed through <code>AudioClient</code> in the SDK. 1
What range should <code>SetVolume</code> use?	Unitree documents volume levels from 0 to 100. 1
What range should each <code>LedControl</code> channel use?	Each RGB channel uses 0 to 255. 1
Why should LED calls be spaced apart?	Unitree notes that the interval between <code>LedControl</code> calls must be greater than 200 ms; the course scripts use conservative one-second waits. 1
What is the speaker ID rule for Day 7?	Use speaker ID 0 for Chinese roles and 1 for English roles, and avoid mixed Chinese/English phrases in a single TTS call. 1
Why is voice-command ASR not the core Day 7 capstone?	ASR adds ambiguity and requires command validation; Day 7 focuses on safe output channels and supervised motion.
What does “single DDS session owner” mean?	Only one operator/script should send commands to the robot during the demonstration. Others observe or assist but do not run competing control scripts.
Why is dexterous hand control excluded?	The Day 7 README explicitly marks dexterous hand control out of scope; manipulation refers to Day 6 arm gestures.
What makes a capstone sequence professional?	It is bounded, announced, visible, logged, recoverable, and uses one command path at a time.

A team is ready to pass Day 7 when it can safely run an audio/LED demo, explain every command in the script, and propose a capstone sequence that respects the single-owner policy and the Day 6 safety boundaries.

20. Instructor Notes for Timing and Classroom Management

Day 7 is engaging because learners can hear and see the robot respond. That engagement can also make the room noisy and less disciplined. The instructor should set expectations before the first TTS call. Only the operator speaks during execution. Observers remain outside the robot's workspace. The group should avoid shouting commands or testing ASR-like behavior unless the instructor explicitly transitions to a future-extension discussion.

The audio volume should be chosen for the room. A volume of 85 appears in the course scripts as a default target, but the instructor may reduce it for smaller rooms or repeated tests. The important point is that the script clamps or bounds volume and prints before-and-after state, not that every classroom uses the same number.

Classroom risk	Mitigation
Students crowd around the robot to hear speech.	Use moderate volume and keep a clear observation boundary.
Multiple teams run scripts at the same time.	Assign a single terminal owner and rotate turns.
Learners edit phrases without changing speaker ID.	Require a language/speaker-ID check before running.
LED demo becomes rapid flashing.	Enforce wait intervals and avoid unnecessary flashing.
Capstone becomes too ambitious.	Limit to one physical action unless instructor approves more.

The session should end with the robot in a known state: LEDs off or in an agreed safe color, no audio playback continuing, no extra scripts running, and the team's evidence saved.

21. Summary

Day 7 completes the G1 portion of the course by adding communication and presentation layers to the robot's existing DDS, locomotion, and arm-control foundation. The most important technical object is `AudioClient`, which provides TTS, volume, playback-related capabilities, and RGB LED control. The most important safety idea is that expressive output does not remove the need for DDS readiness, command ownership, and supervised motion. The most important design idea is that a capstone should be a small, explicit state machine rather than a collection of simultaneously running scripts.

Learners who complete Day 7 should be able to make the robot announce itself, show LED status, perform a bounded gesture, and document the result. More importantly, they should understand why

these steps must be sequenced and supervised. A robot that communicates its intent is easier to trust, but only if its communication is paired with disciplined control.

References